Task 6 – Git Branching Workflow

Git & GitHub Fundamentals (Industry Workflow) | Estimated Time: 1.5 Hours

---

First, What's a Branch?

Imagine your project is a tree. The main branch is the trunk — it's stable, working code. Whenever you want to try something new (a feature, a bug fix, an experiment), you grow a new branch off the trunk. You work on that branch freely without touching the trunk at all. When you're happy with it, you merge it back in. That's branching in a nutshell.

In a real team, every developer works on their own branch. Nobody steps on each other's work, and the main branch always stays clean and deployable.

Step 1 – Create Your Branches

You're going to create three branches — two for features and one for a bug fix. Run these commands from inside your project folder:

```
git branch feature-login
git branch feature-profile
git branch bugfix-readme
```

These branches are now created but you haven't moved to any of them yet — you're still sitting on main. To see all your branches and confirm they exist, run:

```
git branch
```

*The branch with an asterisk (*) next to it is where you currently are. Right now that should be * main.*

Step 2 – Switch Between Branches

Creating a branch doesn't automatically take you there. You need to switch to it. Think of it like opening a different notebook — your other notebooks still exist, you're just working in this one now.

```
git checkout feature-login
```

You're now on the feature-login branch. Any changes you make here won't affect main at all. To jump to another branch:

```
git checkout feature-profile
```

And to go back to main, you can use either checkout or the newer switch command — both do the same thing:

```
git switch main
```

> Pro tip: you can create a branch AND switch to it in one command — git checkout -b feature-login does both at once. Very handy.

Step 3 – Make Changes in Each Branch

Now comes the important part — you'll make different changes in each branch so you can see how they stay completely separate from each other.

Switch to feature-login and add something:

```
git checkout feature-login
echo "Login page feature in progress" >> notes.txt
git add notes.txt
git commit -m "Add login feature notes"
```

Now switch to feature-profile and do something different:

```
git checkout feature-profile
echo "Profile page feature in progress" >> notes.txt
git add notes.txt
git commit -m "Add profile feature notes"
```

And fix the bug on bugfix-readme:

```
git checkout bugfix-readme
echo "Fixed README formatting" >> README.md
git add README.md
git commit -m "Fix README formatting issue"
```

> *If you switch back to main and open notes.txt, you won't see any of those changes. That's the whole point — each branch carries its own work independently.*

Step 4 – Commit Changes Separately

Each branch should have its own commit history. After making changes on a branch, always stage and commit before switching away — otherwise Git might carry your uncommitted changes to the next branch, which gets confusing fast.

The safe workflow every time is:

```
git status          # check what changed
git add .           # stage everything
git commit -m "your message here"   # commit it
git checkout main   # now safely switch
```

Step 5 – See What's Happening Across Branches

Once you've committed on multiple branches, these commands help you see the full picture:

```
git branch
```

Lists all branches. The * shows which one you're currently on.

```
git log --oneline
```

Shows the commit history for the branch you're currently on. Switch to a different branch and run it again — you'll see a different history.

```
git log --oneline --all --graph
```

This is the really cool one. It draws a visual graph of all branches and their commits side by side — you can literally see where branches split off from each other.

What These Commands Actually Mean

- **git branch <name>** — Creates a new branch but keeps you where you are. Nothing moves yet.
- **git branch** — Lists all your branches. The one with * is where you currently are.
- **git checkout <branch>** — Switches you to that branch. Your files update instantly to reflect that branch's state.
- **git checkout -b <name>** — Creates a branch and immediately switches to it — a handy shortcut.
- **git switch <branch>** — A newer, cleaner way to switch branches. Does the same thing as checkout for this purpose.
- **feature-login / feature-profile** — Naming convention: prefix branches with 'feature-' for new features, 'bugfix-' for fixes. This keeps things organised when you have many branches.
- **git log --all --graph** — Visualises the entire commit graph across all branches — great for understanding how your work diverges.